DATE:

HashMap / HashSet

In C++, Hashmap is mainly implemented using unordered_map and HashSet using unordered_set.

1) Hashmap → unordered_map

A Hashmap stores data in key → value form

Syntax :

Ex :

Common operations :

Operation	Code	Avg Time
Insert / Update	mp[key] = value	O(1)
Find	mp.find(key)	O(1)
Check	mp.count(key)	O(1)
Delete	mp.erase(key)	O(1)
Size	mp.size(key)	O(1)

2. HashSet - unordered_set

PAGE No.:
DATE:

A HashSet stores only unique values.

If you insert:

Set will contain:

The duplicate 10 is automatically ignored.

Common Operations:

Operation	Code	Avg Time
Insert	st.insert(x)	O(1)
find	st.find(x)	O(1)
Check	st.count(x)	O(1)
Delete	st.erase(x)	O(1)
Size	st.size(x)	O(1)

3. Main Difference

HashMap	HashSet
Stores:	Stores:
Ex:	Ex:
Useful when you need to associate one thing with another	Useful when you only care about whether something exist or need unique elements

★★★ T.C → O(1), S.C O(n) for both, Worst Case → O(n)

PAGE No.:
DATE:

Important:

Whenever I see ques like
- Does Duplicate exist?
- Have I seen this before?
- Is this element present?
- Count frequency
- find pair with target
- find unique elements

Think about HashSet / HashMap

Questions

1) ~~Two~~ Contains Duplicate:

Input: arr = | 0 | 1 | 2 | 3 |
| 1 | 2 | 3 | 1 |

Output: true

Dry run:
1, 2, 3, 1 i=0
if ✗ insert → 1
if ✗ insert → 2
if ✗ insert → 3
if ✓ true.

```
bool containsDuplicate(vector<int>& nums){
    unordered_set<int> s.t;

    for(int i=0; i<nums.size(); i++){
        if(st.count(x))
            return true;

        st.insert(x);
    }
    return false;
}
```

PAGE No.:
DATE:

2) Valid anagram

Input : s1 = rat, s2 = car.
Output : false.

```
bool areAnagrams (string &s1, string &s2){

    if (s1.size() != s2.size())
        return false;

    unordered_map <int, int> mp;

    for(char c : s1){
        mp[c]++;
    }

    for (char c : s2){
        mp[c]--;

        if(mp[c]<0)
            return false;
    }
    return true;
}
```

Dry run:

s1 = rat , s2 = car

3 != 3 ✗

for s1:

r → 1
a → 1
t → 1

for s2:

c → -1 ✓
a → 0
r → 0

Not Anagram

PAGE No.:
DATE:

3) Two Sum

Input: nums = [2|7|11|5], target = 9
Output: [0,1]

```
vector<int> twoSum(vector<int>& nums, int target) {
    int compliment;
    unordered_map<int,int> mp;
    for(int i : nums) {
        int compliment = target - nums[i];
        if(mp.count(compliment))
        {
            return {mp[compliment], i};
        }
        mp[nums[i]] = i;
    }
    return {};
}
```

Dry run:
[2|7|11|5] i = 0, i = 1
compliment = 9 - 7
= 2
if → mp[0,1] ✓ Ans
mp[2] = 0;

4) Product of Array Except Self

Input: nums = [1|2|3|4] (0 1 2 3)
Output: [24|12|8|6] → return array.

PAGE No.:
DATE:

```
vector<int> productExceptSelf(vector<int>& nums){
    int n = nums.size();

    vector<int> ans(n,1);
    int i;
        // Prefix calculation
    for(i=1; i<n; i++){
        ans[i] = ans[i-1] * nums[i-1];
    }

            // suffix calculation
    int suffix=1
    for(i=n-2; i>=0; i--){
        suffix = suffix * nums[i+1];
        ans[i] = ans[i] * suffix;
    }

    return ans;
}
```

Dry run:

i = 1 2 3



⇒ 1 2 2 6 = ans ← prefix loop ka result



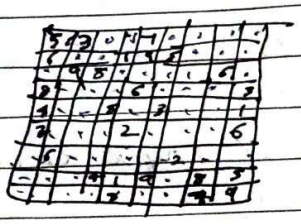
suffix = 1, i = 2 1 0 ⇒ 4 * 3 ⇒ 12 * 2 ⇒ 24

suffix = 1 * 4 ⇒ ~~4 * 3 ⇒ 8~~

PAGE No.:
DATE:

5) Valid Sudoku

Ex 1:



```
bool isValidSudoku(vector<vector<char>>& board){

    unordered_set<string> st;

    for(int r=0; r<9; r++){
        for(int c=0; c<9; c++){

            char num = board[r][c];

            if(num == '.')
                continue;

            int box = (r/3) * 3 + c/3;

            string rowkey = string(1,num) + "in row" +
                            to_string(r);
            string colkey = string(1,num) + "in col" +
                            to_string(c);
            string boxkey = string(1,num) + "in box" +
                            to_string(box);

            if(rowkey || colkey || boxkey){
                return false;
            }
```

6)

```
        st.insert(rowkey);
        st.insert(colkey);
        st.insert(boxkey);
    }
    return true;
}
```

6) Group Anagrams

Input : ["eat","tea","tan","ate","nat","bat"]

Output : [["bat"],["nat","tan"],["ate","eat","tea"]]

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;

    for(string s : strs) {
        vector<int> freq(26,0);

        for(char c : s) {
            freq[c-'a']++;
        }

        string key = "";

        for(int i=0;i<26;i++){
            key += to_string(freq[i]) + "#";
        }
        mp[key].push_back(s);
    }
```

PAGE No.:
DATE:

return freq.strings.end;

~~for (auto it~~

for (auto it : mp)

ans.push_back (it.second);

}

Dry run:

for s: ("eat", "tea", "bat")

{}

s = eat

e - a ⇒ 4 ⇒ freq[4]++

freq[4] = 1

a - a ⇒ 0 ⇒ freq[0]++

freq[0] = 1

+

t - a ⇒ 19 ⇒ freq[19]++

freq[19] = 1

a → 1#

e → 1#

t → 1#

eat will insert

PAGE No.:
DATE:

("eat")

next string s = tea

t = 1 #
e = 1 #
a = 1 #

tea will insert in same

~~k{"eat"}, {"tea"}~~ ("eat", "tea"]

next string s = bat

b = 1 #
a = 1 #
t = 1 #

bat will insert in another group.

So,

[["eat", "tea"], ("bat"]] Ans.

7) Top K frequent Elements

Input: [1, 1, 1, 2, 2, 3], k = 2
Output: [1, 2]

vector<int> topKfrequent(vector<int>& nums, int k) {
unordered_map<int, int> mp;

```
for (int i : nums) {
    mp[i]++;
}

return {int, int>> v;

for (auto it : mp) {
    v.push_back({it.first, it.second});
}

sort(v.begin(), v.end(), [](auto a, auto b){
    return a.second > b.second;
});

vector<int> ans;

for (int i=0; i<k; i++) {
    ans.push_back(v[i].first);
}

return ans;
}
```

Dry run:

Input = 1, 1, 1, 2, 2, 3

1 → 3

2 → 2

3 → 1

PAGE No.:
DATE:

vector v → [(1,3), (2,2), (3,1)]

after sorting → same

i = 0 or i = k → 2 th loop

for i = 0 ⇒ v[0].first → 1
for i = 1 ⇒ v[1].first → 2

[1,2] Ans

8) Longest ~~Sequ~~ Consecutive sequence

Input: nums = [100, 4, 200, 1, 3, 2]
output: 4

```
int longestConsecutive(vector<int>& nums){
    unordered_set<int> st(nums.begin(), nums.end());
    int longest = 0;

    for(int x : nums){
        if(st.count(x-1) == 0){
            int cnt = x;
            int size = 1;

            while(st.count(cnt+1)){
                cnt++;
                size++;
            }
        longest = max(longest, size);
```

PAGE No.:
DATE:

Dry run:

x = 100 ~~200~~ 1

x = 99 == 0 ✓ x = 1 == 0 ✓

curr = ~~100~~ 0 curr = 1
size = 1 size = 1

~~1~~ 101 ✗ 5 ✗

~~2 = 20~~

x = 199 == 0 ✓

curr = 200
size = 1

201 ✗

x = 0 == 0 ✗
size = ~~1 2 3 4~~
curr = ~~1 2 3 4~~ 5
x = 2 ✓
curr = 2 ✓
size = 4 Ans.

PAGE No.:
DATE:

Tricky Code Analysis :

Case 1 : put() Return values & Duplicate keys

~~[illegible]~~

Output :
30
2
30

[["apple", 30], ["banana", 10]] Ans

Case 2 : get() on a Missing key

Output :
~~null~~
NullExceptionPointer
Reason : Missing values hai

Case 3 : HashSet add() Return Value.

Output :

true
false
2 . true

Case 4 : Removing while Iterating

Output :
~~[illegible]~~ One, 1 Three, 3

PAGE No.:
DATE:

Case 6: Comparing values taken from Maps

Output

true
true
true
true

Case 7: Arrays as HashSet keys

Output

false → object banata hai compare isliye.
true

Case 8: Capital letters in a Count Array

output:

Index Out of Bound

Reason: Capital letter in word Java.

Case 9: Printing a frequency Map

Output:

{1=2, 2=1, 3=4}

-1

true

PAGE No.:
DATE:

Case 10 : null keys and null values

Output

2

B

null true

null false

- Code Modification Challenge :

In VS Code